TrustScientist: Evidence and Branch Revalidation for Autonomous Research Agents

Xinyue Fang* Dong Huang† Aoqian Zhang‡ Shuzhi Sam Ge§

Department of Electrical and Computer Engineering,
National University of Singapore,
Block E4, Level 5, Room 42, 4 Engineering Drive 3, Singapore 117583

Abstract

Research agents may continue to follow an earlier decision after the evidence supporting it has changed. Updating a result file alone does not repair that decision or the outputs produced from it. TrustScientist connects evidence checks to recovery of a recorded research workflow. It stores the inputs, policy context, action and checkpoint of each decision. Counterfactual Branch Revalidation (CBR) then rechecks affected decisions in order and rolls back before the first action that changes. Completed checks are saved so that recovery can continue across calls with limited budgets. Under deterministic policies, complete dependencies and stable inputs, we prove agreement with full replay and progress across bounded calls. In 216 controlled update episodes, CBR restores the same checkpoints and final products as full replay with 35.3 percent fewer post-update policy evaluations. A further study of 24 histories with eight consecutive updates produces 2,688 method–event records. With two replay evaluations per call, saving progress increases CBR's completed updates from 148/192 to 192/192. Equally capable generic selective replay matches this behavior. After repairing a shared omission of dummy accuracy, both implementations also match all 192 specified decisions per arm with zero formal policy tokens. On six development projects, a code-generating Agent and local job worker recover all 54 updates per engine. A shared directory-ownership interface prevents in-scope omission failures on both engines, but cannot withdraw an external queue. A frozen transfer to LangGraph tests six previously unused task families, retaining failed Agent analyses in the recovery denominators. Model review still flags unresolved statistical assumptions in two earlier reports. The contribution is a tested software interface for connecting evidence changes, historical decisions and recovery, with explicit limits on what its checks establish.

Keywords: Autonomous agents, decision revalidation, incremental computation, evidence verification, scientific workflows

1 Introduction

A research agent can update its evidence while continuing to act on an old decision. For example, suppose it proceeds because a classifier outperforms a dummy baseline. A revised dataset can reverse that comparison. Recomputing the score updates the evidence, but the earlier decision and

*Corresponding author: xinyue.fang@u.nus.edu; ORCID: 0009-0003-3358-4532.
†Email: dong.huang@u.nus.edu.
‡Email: aoqian@u.nus.edu.
§Email: shuzhisamg@gmail.com.

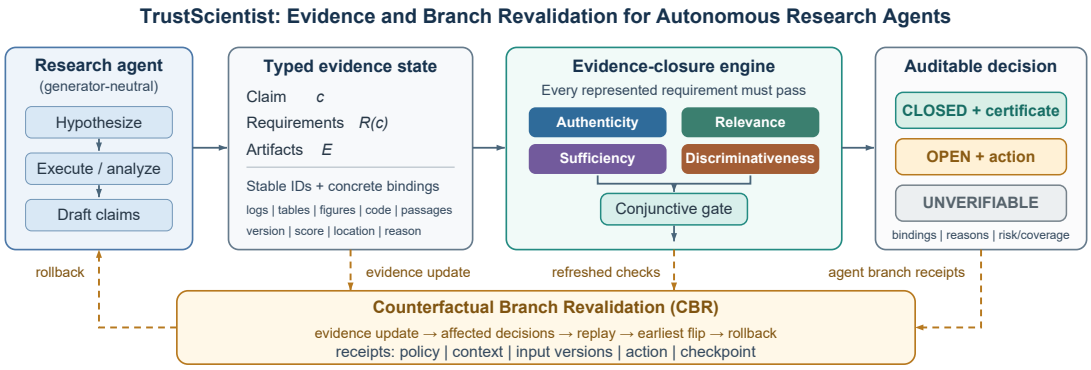


Figure 1. From updated evidence to a valid continuation. TrustScientist checks artifact-bound requirements and records the inputs, actions and checkpoints of agent decisions. After an update, CBR replays affected decisions. It rolls back before the first changed action; otherwise, it keeps the branch and refreshes its version records.

the report produced from it may remain in place. Recovery must identify the affected decision and the outputs that can no longer be used.

Evidence verification asks whether an agent has support for its answer [1–3]. Persistent workflows also need to recover when that support changes: reconsider the affected decisions, choose a restart point and withdraw old outputs. Even an authentic value may refer to the wrong task or metric, so the check's meaning and the recovery action must both be specified.

TrustScientist links these tasks (Figure 1). It checks requirements against identified file versions and records how each decision used them. Counterfactual Branch Revalidation (CBR) reconsiders past decisions under updated evidence using the recorded policy and context. It restores the checkpoint before the first changed action and withdraws registered later outputs; unchanged actions retain the workflow.

The design connects dependency tracking and selective replay to application recovery: the requirement, original decision and recoverable resources form one executable interface. Successful checks persist across budget-limited calls. Generic replay receives the same capability in our comparison.

The paper contributes an executable evidence-to-recovery interface, conditional guarantees of full-replay agreement and bounded-call progress, and controlled tests separating recovery, cost and scientific adequacy. Generated workflows test reports and queued operations under data, metric and code changes. Two engines share an ownership interface; omission probes test its boundary. A frozen LangGraph transfer adds six new-to-project task families and a complete native recovery control. The Technical Appendix contains the full proofs, protocols, development failures and cost records. The current manuscript and recovery/integration source are available at https://github.com/research-artifacts2026/trustscientist; complete frozen execution records accompany this submission package.

2 Related Work

Belief maintenance and reconsideration. Truth-maintenance systems retain beliefs through their justifications [4]. Intention reconsideration studies when an agent should spend computation to revise a commitment [5]. Our question is operational: after a declared evidence change, how should a recorded linear workflow recover? The implementation rechecks historical decisions; it does not choose when reconsideration is worth its cost.

Incremental computation and provenance. Self-adjusting computation establishes consistency for change propagation [6], and build systems separate scheduling from rebuilding [7]. These mechanisms underlie our replay rule. A generic engine can reproduce it when given the same decision records and recovery operations. Provenance models such as PROV-DM record entities, activities and their relations [8]; logical assertions state conditions on program behavior [9]. TrustScientist specifies how these records and checks govern the withdrawal of outputs and restoration of an earlier workflow state.

Persistent Agent frameworks. LangGraph checkpoints persist graph state and support replay from an earlier checkpoint[10, 11]. External reports and queued effects still require application recovery logic. EXT1 therefore compares against a native checkpoint-backed implementation that receives the same cleanup obligations. A bare checkpoint replay diagnostic illustrates the boundary; it is not used to claim superiority over a fully integrated framework.

Verification in research agents. Auditable Autonomous Research links claims to semantic provenance [12]. ScientistOne maintains a Chain-of-Evidence and checks scores, specifications, references and method–code agreement [13]. ClaimGarden is a close proposal for versioned claims and gates on manuscript production and subsequent actions [14]. These works motivate continuous verification. We do not reproduce their complete systems; ClaimGarden's full text was also unavailable for renewed inspection. Our empirical comparison therefore uses the explicit generic-replay control, without asserting superiority to those systems.

Evidence-based judging and iterative verification address support for agent answers [1–3]. Our acquisition comparison holds tools and checks fixed to test the selector. Cost-sensitive hypothesis discrimination has a Bayesian formulation [15]; the selector used here is a heuristic and has no corresponding approximation guarantee.

Scientific support and selective prediction. SciFact [16] and insufficient-evidence fact checking [17] distinguish supported relations from missing evidence. Decomposition can lose meaning [18]. Risk-controlling prediction sets and conformal prediction provide coverage or risk guarantees under stated calibration assumptions [19, 20]. These concerns remain after correct replay: a workflow can consistently execute a scientifically inadequate rule. We evaluate this boundary separately from recovery.

3 Checking Evidence and Recovering Decisions

TrustScientist performs three steps: check the current evidence, reconsider affected historical decisions, and recover from the first changed action. Figure 1 shows the interface; Algorithm 1 gives the recovery procedure.

3.1 Check the stated requirements

A claim c has a mandatory requirement set $R(c)$. Each requirement names the data, code or report it can use and the executable test it must pass. We call these files *artifacts*. Their records include locations, content digests and execution metadata, so a check applies to identified versions rather than to a filename alone.

Checks serve four roles in the original design. Authenticity checks provenance and agreement with the source value. Relevance checks the task, dataset, split, metric or question. Sufficiency

checks specified coverage, and discriminativeness checks stated alternative explanations or comparisons. Each role needs an explicit test; the category name supplies no test by itself.

A requirement is CLOSED when its evidence passes the test, OPEN when an available action can address a missing or failed check, and UNVERIFIABLE when the available representation or verifier cannot decide it. A claim closes only when all its stated mandatory requirements close. The resulting *certificate* records the requirements, file versions, tests and outcomes. It says which specified checks passed, leaving omitted scientific conditions outside its coverage.

The Evidence-Obligation Graph records dependencies among checks. After an update, the system invalidates the affected checks and their descendants, then reruns them in dependency order. Complete dependencies and deterministic tests give the same result as checking everything again. The six-node reporting schema checks access, integrity, task binding, extraction, quotation and semantic adequacy. For unresolved checks whose prerequisites have passed, the acquisition heuristic chooses an affordable action with the largest estimated number of repaired checks per cost unit. It uses a fixed map of tool effects, checks the result after execution, and avoids repeating an unsuccessful action on unchanged inputs.

3.2 Record decisions and replay affected ones

Each decision has a *receipt*: a persistent record of what the agent decided and what would be needed to reconsider it. Receipt $d_i = (\pi_i, x_i, W_i, v_i, a_i, k_i, O_i)$ contains:

- the policy π_i and original context x_i, including retry counters and limits;
- every input the decision depends on, W_i, and its last checked versions v_i;
- the chosen action a_i, preceding checkpoint k_i, and registered outputs O_i produced by the continuation.

The history is linear. Replay uses the original context, so reconsideration does not consume the old retry budget again. Every input that may change must be included in W_i. A fixed model name or temperature zero alone does not ensure that an LLM policy will replay deterministically.

Let $V_E(W_i)$ be the input versions in the current evidence state E. A receipt needs replay, or is *dirty*, when

$$D_i(E) = [V_E(W_i) \neq v_i]. \tag{1}$$

CBR visits dirty receipts in historical order and recomputes each action. If the action stays the same, it saves the checked versions immediately. If the action changes, it sets aside the registered outputs from that decision onward and restores the preceding checkpoint. This prevents later outputs from being reused after their supporting decision has changed.

Saving each successful check matters when a call runs out of budget. The original store could discard this progress and repeatedly check the same prefix. The extension resumes from the remaining dirty receipts. A later input change makes an earlier receipt dirty again through Equation 1.

Incomplete replay or recovery leaves the history blocked. Callers must check this status before recording a new decision or using existing outputs as current results. Missing inputs and unavailable policies also block continuation. The implementation assumes one cooperating writer and inputs that remain unchanged during recovery. It manages registered local outputs; concurrent updates require a snapshot protocol, and undeclared side effects or external jobs require additional recovery support.

Algorithm 1 Progress-preserving CBR for a registered linear history

Require: Receipts $d_{1:n}$, stable evidence E, integer invocation budget $b \geq 1$

Recover pending rollback; verify policy and input versions
for dirty receipt i in historical order **do**
  if $b = 0$ or replay is unavailable **then**
    Persist blocked status and verified attestations;
    return
  end if
  $a'_i \leftarrow \pi_i(E|_{W_i}, x_i)$; $b \leftarrow b - 1$
  if $a'_i \neq a_i$ **then**
    Persist rollback intent and invalidate receipts $i{:}n$
    Quarantine $\bigcup_{j \geq i} O_j$; restore k_i;
    return
  end if
  Persist $v_i \leftarrow V_E(W_i)$, retaining the original action
end for
Persist unblocked status

Owning the effects of a continuation. The generated-workflow study uses a small backend-neutral adapter. Manual integration lists each evidence file and each output path. Managed integration captures the current files in an evidence directory and registers one output directory as a recoverable resource. Report, export, SQLite queue and later files created under that output root are therefore withdrawn together. The caller still supplies the policy, original context, action and restart checkpoint. Both engines receive exactly this interface. New input filenames, writes outside the owned output root and irreversible external actions need additional registration or compensation; directory ownership does not infer missing scientific requirements.

3.3 Guarantees and their assumptions

Assume the original history can be reproduced, every dependency is recorded, policies and context are fixed and deterministic, input bytes remain stable, and registered resources can be restored. Then completed CBR recovery finds the same earliest changed action as full replay. Unaffected decisions reproduce their actions; affected ones are removed from the replay set only after a successful check. Processing them in order therefore misses no earlier change.

With fixed evidence and policies across calls, successful replay and integer budget $b \geq 1$, checking the first r dirty decisions up to a changed action (or all dirty decisions if none changes) takes exactly r policy evaluations and at most $\lceil r/b \rceil$ calls. The Technical Appendix gives the full proofs. These guarantees specialize incremental computation and persistent validation [6, 7]; a generic engine can provide them through the same interface.

Other work remains even when replay is selective. The flat implementation scans $L = \sum_i |W_i|$ dependencies and hashes the inspected bytes on each call. Across q calls, these scans can cost $O(qL)$, in addition to replay, saving progress, rollback and continuation. We therefore measure policy evaluations separately from tokens and elapsed time.

Recovery consistency also leaves two sources of scientific error: a necessary requirement may be omitted, or a stated check may accept inadequate evidence. The appendix separates these sources in a union bound; neither omission risk nor distribution-free component bounds is estimated here. Its proposed calibration guard fails the preset threshold in our study, and static calibration would not establish validity under arbitrary adaptive stopping.

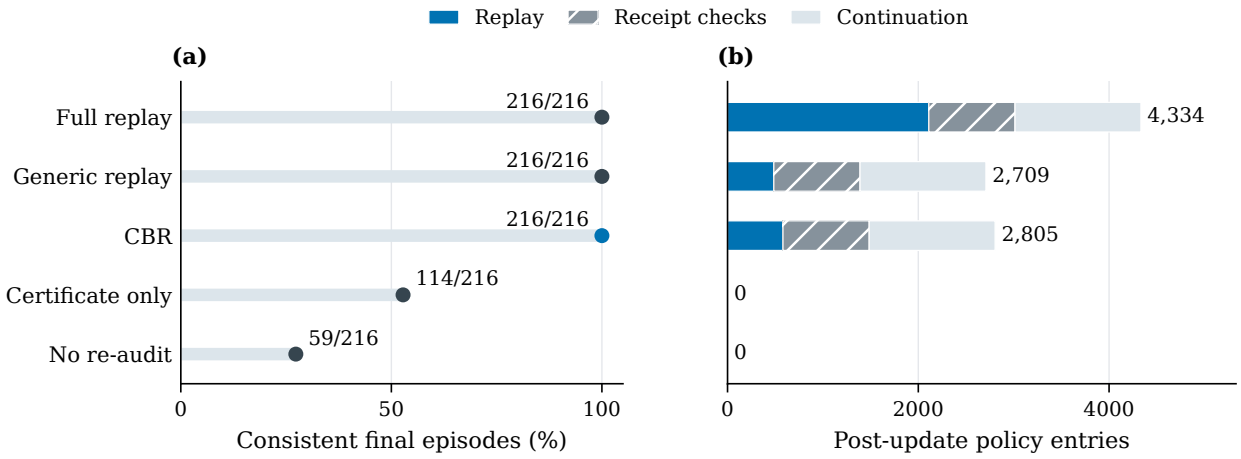


Figure 2. Recovery and replay cost across 216 update episodes. (a) Consistency of final actions and products. **(b)** Policy evaluations for replay, receipt checks and continuation; initialization is excluded. All replay methods recover every episode, while generic replay uses fewer evaluations than CBR. Skipping replay reduces this count to zero but leaves recovery incomplete.

4 Experiments

We evaluate recovery, progress under limited replay budgets, and the cost of enforcing specified decision rules. Separate evidence and report studies test what those rules may miss. The studies use different denominators, which we do not pool. Counts summarize dependent task states; uncertainty estimates use the clusters specified below. The Technical Appendix retains full protocols, detailed costs, development failures and original results.

4.1 Recovery after an evidence update

Setup. We execute histories on Iris, Wine, Breast Cancer and Digits. Three split seeds and search lengths of 4, 12 and 24 decisions give 36 histories. Each fits models and produces a table, chart and report. Six controlled updates per history yield 216 episodes: unchanged content, early and late configuration changes, a training-score substitution, candidate deletion and a revised baseline. The analyses execute on real data, while the policies and update sequence are scripted.

Full replay, CBR and generic flat dependency replay receive the same policy, version checks, output handling, checkpoint restoration and continuation executor. Each performs 135 continuation fits. A separate scorer recomputes macro-F1 from saved predictions, identifies the first changed action and checks the restored state and final outputs.

Recovery and work. All three replay methods recover the exact checkpoint in 102/102 episodes with an action change and consistent products in all 216/216 episodes (Figure 2). Refreshing certificates alone leaves historical actions unchecked. CBR uses 35.3% fewer post-update policy evaluations than full replay (95% paired cluster-bootstrap interval $[32.2, 38.6]$). Two balanced timing repetitions give a smaller wall-time reduction of 10.3% $[9.3, 11.2]$. Both intervals resample 12 dataset–split clusters. Generic replay uses fewer evaluations than CBR: it checks each receipt's current versions, while CBR's union of changed files can revisit some already checked receipts. The saving therefore supports selective replay, without a CBR-specific efficiency advantage.

Table 1 reports the exact recovery counts and post-update policy evaluations, including the

Table 1: Executed controlled histories. Exact counts earliest rollback matches on 102 flip episodes; Final counts consistent final actions and products on all 216 episodes. Calls includes replay, receipt verification and continuation policy entries, excluding common initialization.

Policy	Exact /102	Final /216	Calls
No re-audit	0/102	59/216	0
Certificate-only	0/102	114/216	0
Full replay	102/102	216/216	4,334
Generic replay	102/102	216/216	2,709
TrustScientist-CBR	102/102	216/216	2,805

Table 2: New consecutive-update experiment: 192 events per method and arm. Consistent requires completion, current actions and matching products. Calls are all post-update policy entries; all successful replay methods perform 96 actual continuation fits per arm.

	Unbounded		Budget two	
Method	Consistent	Calls	Consistent	Calls
Full replay	192	3,304	120	6,336
Generic replay	192	2,532	148	4,832
Original CBR	192	2,532	148	4,832
Progress CBR	192	2,532	192	2,532
Generic + progress	192	2,532	192	2,532
Full + progress	192	3,304	192	3,304
Certificate-only	104	0	104	0

certificate-only and no-re-audit controls. Counts in the table use the original executed controlled-history protocol rather than the later generated-workflow cohorts.

4.2 Recovery across repeated updates

We apply eight consecutive changes to each of 24 persistent histories and compare seven methods under two budgets: unlimited replay or at most two replay evaluations per call. This gives $24 \times 8 \times 7 \times 2 = 2{,}688$ method–event records. The four reused dataset families and three split seeds form 12 dependence clusters. Each method starts with the same histories and updates, although histories can diverge after a failure. Generic replay and full replay each have a resumable control that saves successful checks, allowing us to separate selectivity from saved progress.

With budget two, original CBR and generic replay complete 148/192 updates. Saving progress raises both to 192/192; resumable full replay also completes 192/192 (Figure 3). With sufficient budget, all replay methods complete 192/192. Thus retaining completed checks repairs the repeated-prefix failure for both selective and full replay.

CBR and generic replay with saved progress each use 2,532 post-update policy evaluations, 23.4% fewer than full replay's 3,304. They also tie under budget two; the original variants spend extra work repeating checks. Totals include replay, receipt checks and continuation, excluding the common 864 initialization evaluations per method and budget setting. These are local deterministic policy calls. Token costs and detailed hashing, persistence and timing measurements are separate quantities.

Table 2 gives the per-method totals. Nonresumable full replay completes only 120/192 events under budget two, compared with 192/192 after saving progress.

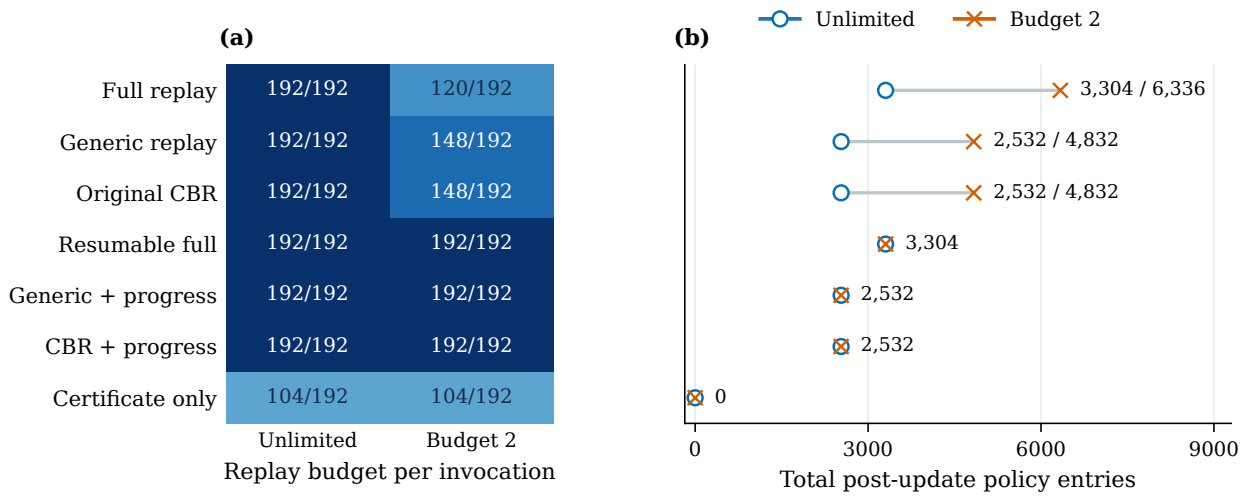


Figure 3. Recovery under repeated updates and limited replay budgets. Each method processes 192 events (24 histories, eight updates). **(a)** Consistent completions. **(b)** Policy evaluations, excluding initialization: circles denote unlimited budget; crosses denote two replay evaluations per invocation. Paired labels show unlimited/budget-two totals; equal markers overlap. CBR and generic replay tie when both retain progress.

4.3 Decision-rule repair and execution costs

Repair and comparison. Earlier eight-task development rounds exposed uncertainty-analysis defects and a shared omission of dummy accuracy from two classification contracts. The corrected V5 comparison requires both accuracy and macro-F1 with their dummy comparators in both deterministic arms. The classifier outputs reproduce on the fixed validation splits. Two legacy task-description lists still omit dummy accuracy; the accompanying erratum distinguishes these lists from the corrected runner, common contract and actual outputs. Failed development attempts remain outside the successful formal observations.

The frozen comparison uses eight development tasks, three new controller seeds, eight states and three arms: 576 records, or 192 per arm. The states are initialization, an unrelated note, expanded data, uncertainty, cosmetic code change, scope, controlled corruption and repaired execution. Each arm initializes independently and performs four analyses and eight verifier calls per task/seed; arm order rotates. Inputs, requirements, release checks and decision granularity are shared.

The principal control, generic finite dispatch, directly evaluates the same registered decision rule as CBR. The secondary Qwen3-8B control considers changed requirements separately using model deliberation; the figures call it the *model guard slice*. This second comparison measures rule execution versus model deliberation, whereas the generic control tests CBR against another implementation of the same rule.

Decision outcomes. Both deterministic arms match the specified rules on 192/192 states: 168 valid states and 24 injected faults. Their analyses, decision triples and rendered reports are identical. The model arm matches on 153/192: 129 valid states and all 24 faults (Figure 4). Its 39 errors reject valid states; no unsafe release occurs under the finite contract. Here, *finite-contract conformance* means agreement with the stated decision rules. It does not establish scientific correctness, and the new seeds do not make these reused tasks held out.

Table 3 gives the exact V5 corrected-contract denominators. It supersedes, rather than silently reinterprets, the earlier V3/V4 development outcomes retained in the Technical Appendix.

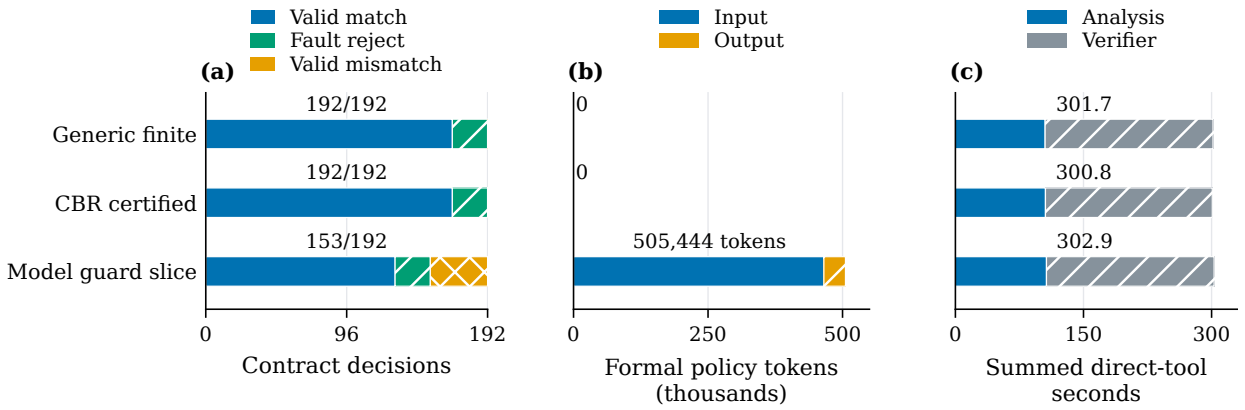


Figure 4. Finite-contract decisions and measured execution costs. Each arm has 192 states across eight developmental tasks and three seeds. **(a)** Decisions matching the specified contract; mismatches reject valid states. **(b)** Formal policy tokens, excluding setup, development and model review. **(c)** Summed analysis and verifier time, including initialization; this is not end-to-end latency. Contract conformance does not establish scientific correctness.

Table 3: V5 after the shared dummy-accuracy repair. Release and joint conformance coincide. The endpoint is a public finite contract, not scientific ground truth. All 192 planned records per arm are present.

Method	Contract correct	Injected faults rejected	Policy tokens
Generic finite dispatch	192/192	24/24	0
CBR certified	192/192	24/24	0
Model guard slice	153/192	24/24	505,444

Measured costs. Both deterministic arms use zero formal policy tokens. Their absolute difference is zero; a percentage saving against zero is undefined. CBR reuses 48 exact decision proofs and computes 120, while generic dispatch computes 168 without certificate memoization. Ordinary memoization can reproduce this reuse, so the generic control is not the strongest possible selective replay engine.

Every numerical verification still runs: 96 analyses and 192 verifier calls per arm, including initialization. CPU and peak-memory differences are small and establish no CPU advantage. Summed direct-tool seconds can overlap and are not end-to-end latency. Table 4 gives the per-arm execution and policy costs.

The model arm makes 345 actual requests with 138 application cache hits and no transport, parsing or missing-usage failures. It uses 465,293 input and 40,151 output tokens, totaling 505,444 formal policy tokens. Adding 2,597 shared setup tokens gives 508,041 measured local-model tokens. API latency sums to 1,029.82 seconds. The sampled GPU board consumes 0.04933 kWh during the formal window; this excludes startup, overnight idle, development and whole-host energy, and cannot be assigned to arms. GPT-6 review and development token/charge telemetry is unavailable, rather than zero. The appendix retains environment versions, logical I/O limits and failed preflight costs.

4.4 Generated Agent workflows and integration omissions

Setup. The final generated-workflow round (WF3) uses Qwen3-8B to inspect public data, write and execute analysis code with a bounded repair loop, and draft a report. A local worker then

Table 4: Measured V5 costs, including each arm's initialization. Wall seconds are sums of direct tool executions and can overlap across jobs. CPU and RSS cover direct tools, not the entire controller or model service.

Measure	Generic dispatch	CBR	Model guard slice
Actual analysis/verifier calls	96/192	96/192	96/192
Analysis wall seconds	105.08	105.46	106.65
Verifier wall seconds	196.59	195.35	196.29
Direct user+system CPU seconds	338.76	337.30	339.58
Peak direct RSS (KiB)	330,624	331,396	330,612
Actual policy API requests	0	0	345
Prompt/completion tokens	0/0	0/0	465,293/40,151
API latency sum (s)	0	0	1,029.82
Persistent workspace (MB, decimal)	215.67	215.67	219.35

Table 5: Workflow recovery after data, metric and code changes (WF3). Each row covers 54 planned and attempted updates. A recovered update has current numerical results, report, export, queue and follow-up records.

Engine / integration	Recovered	Stale jobs run
TrustScientist / owned directory	54/54	0/54
Generic / owned directory	54/54	0/54
TrustScientist / explicit paths	54/54	0/54
Generic / explicit paths	54/54	0/54
No recovery	0/54	54/54

consumes a real SQLite job and writes a downstream result. The six reused research projects cover cardiac measurements, walking, memory, preregistration, omics and metabolic classification. Three seeds give 18 histories. Each undergoes expanded data, an added metric and an executed code defect, in that order. The defect adds 0.25 to a required estimate. These changes and the release rule are controlled; code and report text are genuinely model-generated.

We cross two recovery engines (TrustScientist and an independently written generic engine) with two integrations: explicit registration of four output paths, or the same shared directory-ownership interface. A fifth arm skips recovery. Inputs, checks, checkpoints and Agent responses are paired. The actor cannot read verifier answers. A separate calculation checks required estimates, saved predictions and downstream arithmetic. A successful recovery must withdraw old outputs and pending jobs, restore the analysis checkpoint, and produce current report, export, queue and follow-up records. This checks machine-readable report contents and provenance, not every prose claim. Each arm has 54 planned updates. Table 5 reports recovery and stale-job execution.

Recovery and omissions. All 18 initial analyses pass the specified numerical checks. Both engines restore all 54 updates with either complete integration. For example, expanded metabolic data change validation accuracy from 0.942928 to 0.945409; the old queued analysis is withdrawn before the worker can consume it. Adding balanced accuracy then updates the report and export, and the injected code error is rejected before a regenerated analysis resumes. Without recovery, all 54 old jobs execute.

We separately omit each of four output registrations, omit a data dependency, or move the queue outside the owned directory. Each probe has 18 histories per engine and integration. For the first five probes, explicit registration leaves old effects or misses the update (0/90 safe withdrawals

per engine). Directory ownership withdraws all in-scope effects (90/90 per engine). For the external queue, both integrations fail in 18/18 cases and the worker consumes the old job. The managed probes test containment: there is no individual output-registration statement to delete. They do not measure natural developer error probabilities or equivalent source mutations.

Software value and limits. The same task-independent interface runs on both engines and all six tasks. It replaces four explicit output paths with one owned output root; this is a structural integration measure, with no developer-time study. Each recovery arm uses 96,861 logical Agent tokens, including initialization and continuation requests; the no-recovery arm uses 26,855. Shared caching yields 66,497 actual formal tokens across the experiment, not a per-engine saving. WF3 report prose still includes an unsupported significance statement for omics; numerical recovery does not validate that claim. These are reused development tasks. The preceding round passed only 7/18 initial analyses; all failures remain archived. Public-data inspection and better tool feedback were then added uniformly, and all tasks were rerun. This change improves the Agent, not the replay engine. The appendix reports both rounds, an earlier isolation failure, full measured costs and remaining report-review requirements. The supported contribution is reusable recovery integration with an explicit ownership boundary.

4.5 Frozen integration in LangGraph

Protocol. EXT1 transfers the WF3 interface and both replay engines unchanged to LangGraph 1.2.16 with its SQLite checkpointer. An author-built graph runs data inspection, code generation, isolated execution, bounded repair, report generation and a local queued operation. Recovery uses the framework's replay API to resume a real pre-analysis checkpoint [10, 11].

Six families absent from recorded development registries use public CO2, sunspot, Nile, Grunfeld, copper and macroeconomic data from statsmodels 0.15.0 [21]. Three seeds per family give 18 histories; the three WF3 updates give 54 planned slots per arm. Inputs, numerical rules and code are frozen before formal requests. TrustScientist, generic replay and a complete native LangGraph backend each use both integrations: an owned output directory or explicit output paths. The native backend stores receipts in graph state and explicitly withdraws application resources. All six arms share information, policies, outputs, checkpoints, Agent responses and a three-attempt coding budget. These are paired configurations, not six independent task samples. Bare checkpoint replay omits application cleanup and serves only as a diagnostic.

Completion, withdrawal and omissions. Table 6 separates end-to-end completion from recovery conditional on an attempted update. Only 5/18 initial analyses pass the numerical and product checks. The 13 failures prevent 39 updates; two later regeneration failures prevent three more. Thus each arm attempts 12 of 54 planned slots, leaving 42 unexecuted. Each complete backend recovers 10/12 attempts (10/54 planned): nine Nile updates and one copper update. All 12 attempts withdraw stale effects before consumption; the two failed regenerations then stop. No stale-job outcome is observed for the 42 unexecuted slots. Failures include incorrect result structure and regression code. The frozen cohort is neither repaired nor rerun after inspection.

Under controlled omissions, directory ownership protects all 25 executable in-scope cases per backend; explicit registration protects none. Both fail all five executable outside-root queue cases. This tests resource containment, not natural developer-error probability. Framework glue has 26 nonblank, non-comment lines (including docstrings), versus 45 for the native baseline. The normal-orchestration patch adds nine lines and removes three, excluding glue and tests. No frozen engine or interface line changes; this source inventory does not measure human effort.

Table 6: LangGraph transfer (EXT1): completion and resource withdrawal. Recovery requires both current numerical products and correct resource handling. Counts in panel (a) are per configuration: each complete backend has identical results with owned-directory and explicit-path integration; the configurations are not pooled.

(a) Complete integrations and checkpoint-only diagnostic

Backend	Recovered / planned	Recovered / attempted	Stale jobs / attempted
TrustScientist	10/54	10/12	0/12
Generic replay	10/54	10/12	0/12
Native LangGraph	10/54	10/12	0/12
Checkpoint only (diagnostic)	0/54	0/12	12/12

(b) Safe withdrawals under controlled integration omissions

Integration	In-scope probes safe / executed	Outside-root queue safe / executed
Owned directory	25/25	0/5
Explicit paths	0/25	0/5

Panel (b) is per complete backend. Five in-scope omissions have 90 planned probes per integration; only 25 execute. The outside-root probe has 18 planned cases; only five execute. Unexecuted cases remain unobserved.

Costs and interpretation. Every arm, including the diagnostic, uses 85,386 logical tokens for initialization, failures and retries. Shared responses and caching yield 80,809 actual formal tokens across the experiment, plus 6,288 for setup and smoke tests. Logical totals are not independent per-arm inference bills. The complete backends tie: EXT1 supports transfer of the recovery interface to one framework, with protection against in-scope omissions. The 10/54 end-to-end completion count and concentration in Nile and copper expose limited Agent generalization. These data establish neither a CBR-specific recovery or token advantage nor scientific validity, production reliability or independent-developer productivity.

4.6 Scientific report review

Before V5 execution, we selected all eight tasks, the first controller seed and four substantive states for CBR and the model arm, giving 64 reports. Two fresh GPT-6-Astra contexts reviewed six dimensions using public data, saved predictions, bootstrap records and provenance. Method names and automatic scores were hidden and report order shuffled, although writing style could reveal the source. The reviewers used separate contexts from the same model family; their separation was instructional. The 128 judgments are diagnostic model assessments, with no human labels. A report is invalid if any dimension is invalid, and otherwise uncertain if any dimension is uncertain.

The reviewers jointly accept 30/32 CBR reports and 22/32 model reports (Figure 5). They agree on 62/64 overall labels ($\kappa = 0.896$). Of the 32 paired states, 22 are jointly valid for both sources, eight only for CBR, and two for neither. Generic finite-dispatch reports are identical to CBR and were not reviewed separately. The result therefore provides no CBR-specific quality advantage over generic dispatch or noninferiority guarantee on unseen tasks.

The two uncertain CBR reports concern omics uncertainty and scope. Gene-pair Spearman estimates and resampling arithmetic reproduce, but gene-row exchangeability and dependence are not established. The percentile interval is consequently not independently justified as calibrated uncertainty. This scientific assumption lies outside what the finite contract checked. The six

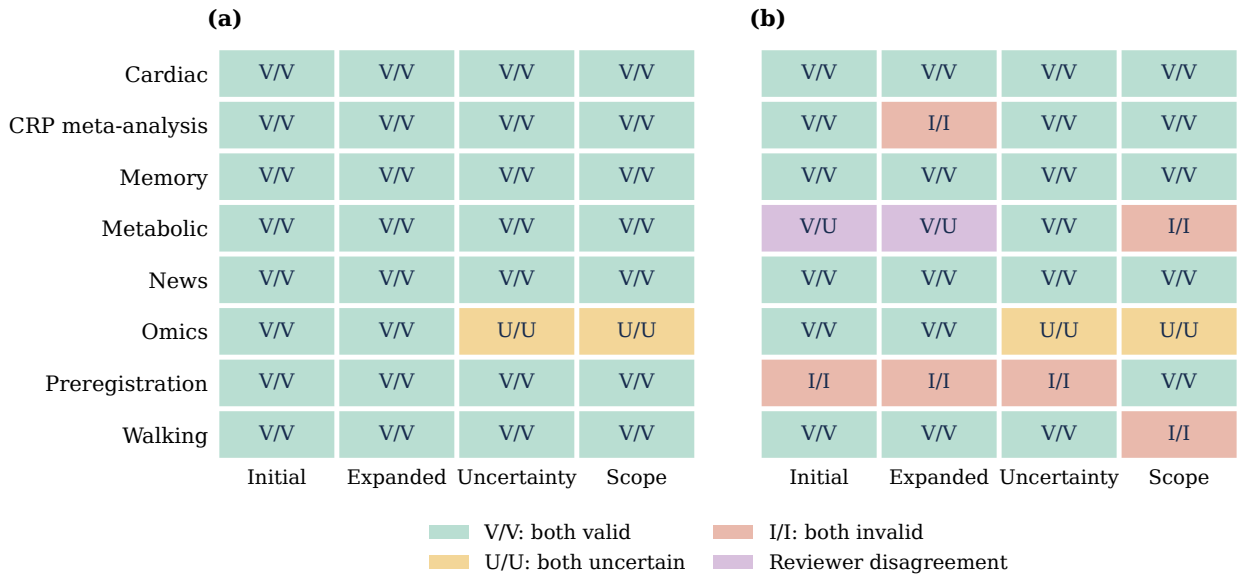


Figure 5. Two GPT-6 reviewers' judgments of 64 scientific reports. (a) CBR reports. **(b)** Model guard-slice reports. Each cell shows reviewer 1/reviewer 2: valid (V), invalid (I) or uncertain (U). Both sources remain uncertain on two omics cases. Generic finite-dispatch reports are identical to CBR and were not rated separately. These are same-family model judgments, with no human labels.

jointly invalid model reports include an incorrect interval interpretation, an unsupported dummy comparison, blocking of descriptive counts without justification, and an unrelated rationale. Two more reports have disputed "significantly higher" wording. All judgments and disagreements remain archived without rerating cases based on their outcomes.

4.7 Evidence maintenance and scientific limitations

Maintaining checks after file changes. The original CORE-Bench study applies nine controlled events to each of 45 capsules, using 124 source files behind 79 reporting states. All methods receive the same dependencies, change identifiers, checker and sufficient budget. Across 711 certificate decisions, graph and flat tracking each retain 332 valid certificates and reject 163 stale ones. They match full revalidation with 1,530 rather than 2,888 checks, a 47.0% reduction (capsule-bootstrap 95% interval $[42.2, 51.8]$). The stateful Qwen3-8B validator accepts 38/163 stale certificates and loses five valid closures. The graph/flat tie in Figure 6 attributes the benefit to executable dependencies rather than the graph representation.

Table 7 provides concrete saved cases showing a successful branch rollback and a failed semantic metric binding. These are selected illustrations, not independently sampled error rates.

Checking requirements and acquiring evidence. Reference-assisted retrieval finds 62/79 CORE requirements; 17 stay open. On 62 nominal-clean and 310 constructed corrupted states, TrustScientist closes 62 and zero; GPT-5.6-Sol closes 59 and zero; DeepSeek-V4-Flash closes 55 and 149. Each receives the same supplied predicates. The three GPT nonclosures expose task, metric or context mismatches missed by the supplied relation flags, so nominal clean is not semantic ground truth. Restoring known clean records closes all 310 controls; this tests restoration rather than autonomous repair. A separate five-run fresh-workspace diagnostic succeeds on 19/20 tasks and 58/59 questions per run, retaining the incompatible CUDA task in every denominator.

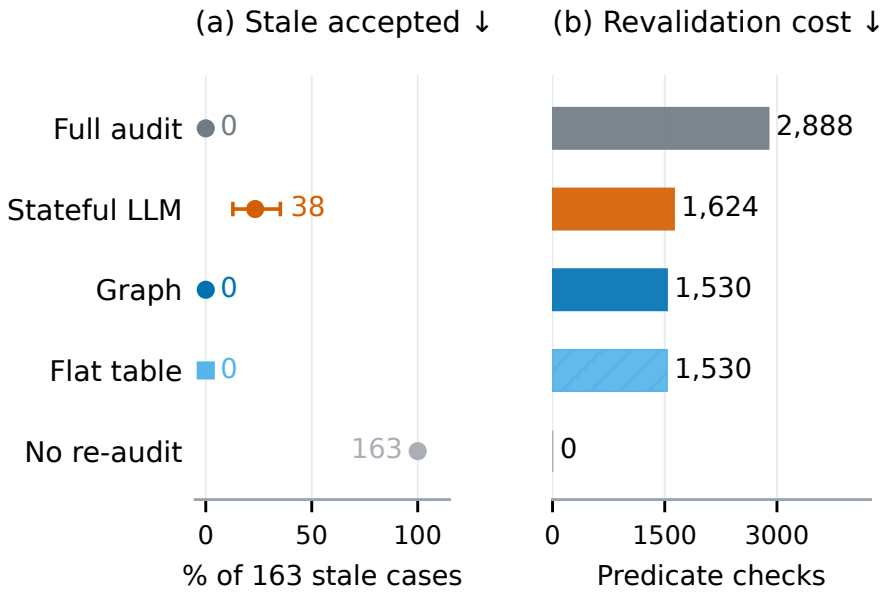


Figure 6. Certificate maintenance across 405 controlled updates. (a) Stale certificates accepted out of 163; whiskers show capsule-bootstrap 95% intervals. **(b)** Revalidation checks, excluding initialization. Graph and flat tracking both reject all stale certificates with fewer checks than full re-audit. The study covers 45 capsules and 711 certificate decisions.

With shared tools and budget four, the unguarded acquisition selectors tie on every paired outcome: 21/46 correct natural closures with nine false terminations, and 22/36 correct controlled closures with two false terminations. Calibration accepts seven reserved capsules, including two errors. Its one-sided 95% upper bound, 0.659, exceeds the preset 0.10 threshold, so the guard allows zero useful closures. A disjoint 41-question pilot also ties at 14 correct and 11 incorrect closures per method, while the heuristic uses 4.4% more tokens. Table 8 gives the coverage denominator and Table 9 details agreement on corrupted states. Table 10 preserves the acquisition ablation outcomes, including the graph-removal and re-audit failures. These are separate from V5 finite-rule conformance and the generated-workflow evaluations. The appendix retains budget ablations, failures and complete costs.

Checking scientific meaning. On 79 fixed reports, adding scope checks to the blind-answer gate reduces correct closures from 34 to 25, while wrong closures fall from six to five. With fixed retrieval on SciFact, TrustScientist correctly resolves 98/209 evidence-bearing pairs and falsely closes 11/130 no-evidence pairs. Its 118 closures also contain nine SUPPORTS/REFUTES reversals. The matched-coverage curve in Figure 7 excludes those reversals from its no-evidence false-closure measure. Eight of nine reversals occur on rationale misses; this post hoc association does not establish that changing retrieval would fix them. Complete operating points and saved repetitions remain in the appendix.

Table 7: Recorded examples from the evaluated studies, selected after inspecting saved outputs. Version and branch changes are controlled interventions; acquisition reads published artifacts. The cases illustrate recovery, maintenance, and a binding failure; they provide no frequency estimate. Source IDs and receipts are in the Technical Appendix's Recorded Case Studies.

Case	Evidence change and TrustScientist response	Paired comparator and boundary
Two-step recovery SVM F1 on Musk1+	Wrong capsule binding and interrupted extraction keep the report OPEN. Rebind, then resume extraction; re-audit closes the unguarded report with the quoted answer 87.5155.	The structured selector also closes correctly after two actions. Risk-guarded ECC remains OPEN because calibration fails.
Shared-source update Three reporting chains	A changed source invalidates all three downstream certificates. Dependency re-audit checks ten affected nodes and removes all stale closures.	The stateful Qwen validator updates one chain, retaining two stale closures. Flat tracking matches the graph; full re-audit checks eleven nodes.
Historical branch flip Two recorded decisions	Replay leaves the earlier REFINE unchanged, but changes the later PROCEED to REFINE. CBR invalidates the later continuation and rolls back before it.	Full replay chooses the same checkpoint. The certificate-only tracker updates predicates but retains the old PROCEED. No replacement experiment is executed.
Wrong-metric binding Random-classifier macro-F1	The file, digest and supplied relation flag pass, although the quoted value belongs to neural-network macro recall. TrustScientist incorrectly closes.	GPT-5.6-Sol keeps the manifest OPEN and identifies the metric and classifier mismatch.

Table 8: Reference-assisted candidate retrieval. Field and language panels describe the same 79 requirements; they are not additive.

Subset	Candidates/total	Coverage (%)
Computer science	29/36	80.6
Social science	21/24	87.5
Medical science	12/19	63.2
Python	34/42	81.0
R	28/37	75.7
Overall	62/79	78.5

Table 9: Matched-manifest compliance on retrieved candidates. Lower corrupted closure is better. Each policy receives the same evidence fields; the LLMs use their saved fixed-prompt configurations.

Policy	Nominal clean closed	Corrupted states closed
DeepSeek-V4-Flash	55/62	149/310
GPT-5.6-Sol	59/62	0/310
TrustScientist	62/62	0/310

Table 10: Candidate outcomes before the risk guard, budget 4. Columns show correct and incorrect candidate closures; denominators are all 46 natural or 36 controlled attempts. ECC's final risk-qualified closures are zero. Only w/o risk guard and Structured LLM turn these candidates into final closures. The postcheck-change scenario is included in controlled states.

	Natural ($n = 46$)		Controlled ($n = 36$)	
Policy	Correct	Wrong	Correct	Wrong
ECC	21	9	22	2
w/o graph	21	9	11	1
w/o risk guard	21	9	22	2
w/o budget selection	21	9	22	2
w/o re-audit	18	4	0	12
Structured LLM	21	9	22	2

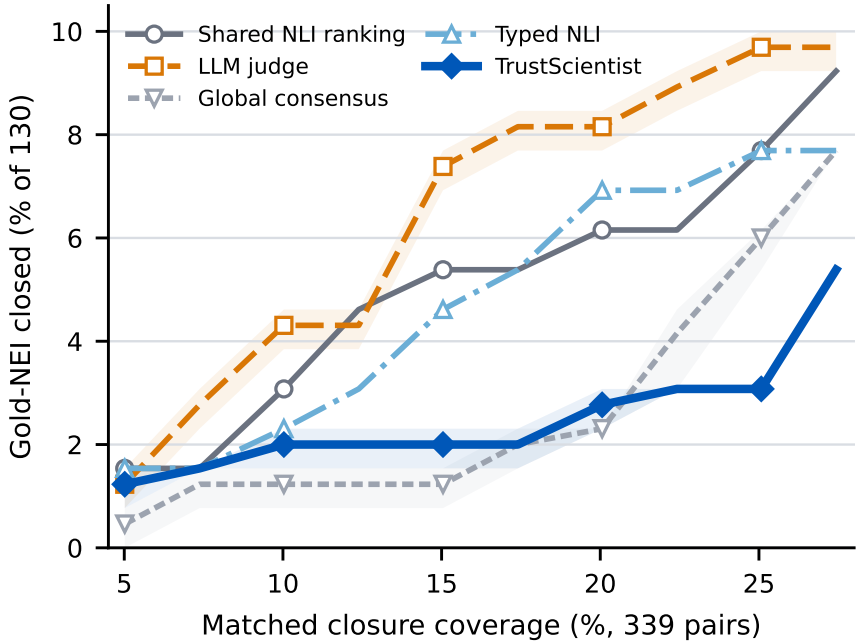


Figure 7. False closure at matched coverage on SciFact. The vertical axis measures accepted no-evidence pairs out of 130; SUPPORTS/REFUTES reversals are excluded. Curves show five-run means; shading spans runs for LLM rankings. Shared NLI groups the direct, rule-based and risk-controlled NLI policies. Tied scores retain dataset order.

5 Discussion and Limitations

5.1 What the interface adds to selective replay

Selective replay provides the mechanism for rerunning computations affected by an update. TrustScientist specifies how to use that mechanism to recover a research workflow. A dependency alone identifies where a change may matter; recovery also needs the original decision rule, context, checkpoint and ownership of later outputs. Table 11 separates these requirements from the mechanisms that implement them.

The executable contract connects decisions to recoverable resources. Unchanged actions retain

Table 11: Recovery requirements and their relation to selective replay.

Recovery task	Information or behavior required	Relation to generic replay
Find affected decisions	Complete input dependencies and checked versions	Standard dependency tracking; shared by both implementations.
Detect a changed action	Original policy, context and action; checks in historical order	A generic engine can evaluate the same decision targets.
Restore the workflow	Preceding checkpoint and registered outputs to withdraw	Supplied by the application recovery interface; shared in our comparison.
Resume after budget exhaustion	Save successful checks and invalidate them after new changes	Conventional persistent progress; generic replay matches CBR.

the workflow; a changed action triggers rollback and withdrawal, while incomplete recovery blocks continuation. The appendix illustrates why metric comparisons, historical thresholds and output ownership matter. Its argument requires identical observable information, including permitted queries, in states needing different responses; it does not establish a general lower bound or minimal record.

Generic replay matches CBR with the same interface and saved progress. V5 already ties on tokens without generic certificate memoization; ordinary memoization can also reproduce CBR's proof reuse. WF3 and EXT1 instead show an interface benefit: owning one directory protects in-scope resources against omissions from per-output lists. This benefit is shared by independently implemented and native LangGraph backends. Transfer of unchanged code supports compatibility with one external framework; developer effort, independent applications and production maintenance remain unmeasured.

5.2 Limits of the evidence

Updates, policies and release thresholds are controlled. Generated workflows add model-written code and reports, checked by a deterministic rule and a bounded local worker. Four reused dataset families and three seeds form 12 dependence clusters, not 12 domains. V5 reuses eight development tasks and fixed checkpoints; new seeds are not held-out tasks. EXT1 adds six previously unused families, retaining substantial initial failures. Its 18 seeded histories are not 18 domains, and pretraining exposure is unknown. Distributed recovery and naturally evolving autonomous decisions remain untested.

Recovery requires complete dependencies, reproducible policies and restorable resources. Scientific adequacy additionally requires adequate checks. The dummy-accuracy repair and unresolved omics assumptions expose this distinction. Requirements and reference-assisted bindings lack an independent expert completeness audit. A separate scorer checks the same specified rules; GPT-6 report judgments provide no human ground truth. Workflow provenance and required values are checked, but narrative reasoning has no completed human assessment. Outside-root jobs still execute in every corresponding controlled probe.

Acquisition selectors tie, added semantic checks lose useful closures, and calibration permits none. Zero errors at zero coverage do not establish usable risk control. Fewer policy evaluations need not save tokens or elapsed time when hashing and persistence dominate. V5 records local-model and direct-tool costs; development, review and whole-host costs remain incomplete. These studies establish neither general scientific accuracy nor an exclusive advantage over capable selective replay.

6 Conclusion

TrustScientist connects evidence checks to recovery of past agent decisions and their registered outputs. Under complete dependencies, fixed deterministic policies and stable inputs, ordered replay recovers the same first changed action as full replay, while saved checks allow progress across bounded calls. Controlled experiments support these properties and show that equally capable generic replay can achieve them too. The generated-workflow study further demonstrates recovery of reports and queued local operations on two engines, with a shared ownership interface that prevents in-scope registration omissions. Frozen integration with LangGraph extends the test to new task families. Only 10/54 planned updates complete, although 10/12 attempted updates recover; limited Agent generalization constrains this result. The interface makes recovery behavior explicit and testable; scientific validity still depends on the requirements and checks supplied to it. The shared dummy-accuracy repair and unresolved omics assumptions show why these two forms of correctness must be assessed separately.

Statements and Declarations

Funding. The authors received no funding for this study.

Competing interests. The authors declare no competing interests.

Author contributions. Xinyue Fang carried out the main research work. Dong Huang, Aoqian Zhang and Shuzhi Sam Ge contributed to methodological discussions. All authors agreed to submission.

Data and code availability. S1 is the Technical Appendix. S2a–S2k preserve eleven frozen archives: original recovery/evidence (S2a–S2b), diagnostic/repaired pilots (S2c–S2d), optimization rounds (S2e–S2g), earlier model review (S2h), and corrected V5 source, execution and review (S2i–S2k). The V5 metadata erratum accompanies them. S3 contains WF1–WF3 source, input hashes, raw Agent responses, executed code, SQLite effects, independent checks and blank human-review forms. S4 contains EXT1 freezes, inputs, checkpoints, all planned outcomes, costs, audit evidence and an uncompleted independent-developer protocol. The Submission Index supplies exact filenames, roles and SHA-256 checksums. Older manuscripts inside these archives are historical; this paper and S1 are current. WF3 is distinct from optimization V3.

Dataset provenance is retained for scikit-learn, the identified public capsules and statsmodels; restoration instructions cover separately distributed CORE witness binaries. The public repository https://github.com/research-artifacts2026/trustscientist provides the current manuscript and recovery/integration source; S2–S4 provide the full frozen execution evidence for the reported study rounds. Third-party licenses and notices remain applicable. New source is supplied for peer review without an additional public reuse license; request reuse permission from the corresponding author.

Ethics and consent. This study uses existing public benchmark datasets and does not recruit participants or collect new participant data.

AI assistance. An OpenAI Codex assistant inspected the manuscript and code, generated substantive text and code revisions, executed the new controlled experiments and a Qwen3-8B

generated-analysis pilot through the authors' environment, and prepared the scoring and submission materials. This assistance exceeds copy editing. The scientific report review uses two fresh contexts of GPT-6-Astra at ultra reasoning effort; these are model judgments with no human labels. The listed human authors must verify the results and accept responsibility for the final manuscript; the assistant is not an author. Prior assistance disclosures remain with the archived material.

Supporting Information

S1 (`TrustScientist_Technical_Appendix.pdf`) preserves full methods, conditional proofs, protocols, study denominators, development failures, cost audits and original figures. S2a–S2k retain earlier execution evidence and model-review records. S3 supplies the generated-workflow study in Section 4.4, including unsuccessful rounds and all planned denominators. The Submission Index lists these files and their roles. S4 supplies the frozen LangGraph transfer in Section 4.5, including failed analyses and omission probes. Blank human-review forms are available for optional independent assessment; they are not completed human evaluation. The original figures and experimental records remain unchanged.

References

[1] Laksh Advani. From confident closing to silent failure: Characterizing false success in LLM agents, 2026. URL https://arxiv.org/abs/2606.09863. FAGEN workshop at ICML 2026.

[2] Leyao Wang, Yanan He, Peng Chen, Asaf Yehudai, Yixin Liu, Rex Ying, Michal Shmueli-Scheuer, and Arman Cohan. Time to REFLECT: Can we trust LLM judges for evidence-based research agents?, 2026. URL https://arxiv.org/abs/2605.19196.

[3] Yuxuan Wan, Tianqing Fang, Zaitang Li, Yintong Huo, Wenxuan Wang, Haitao Mi, Dong Yu, and Michael R. Lyu. Inference-time scaling of verification: Self-evolving deep research agents via test-time rubric-guided verification, 2026. URL https://arxiv.org/abs/2601.15808.

[4] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979. doi: 10.1016/0004-3702(79)90008-0.

[5] Martijn Schut, Michael Wooldridge, and Simon Parsons. The theory and practice of intention reconsideration. *Journal of Experimental & Theoretical Artificial Intelligence*, 16(4):261–293, 2004. doi: 10.1080/09528130412331309277.

[6] Umut A. Acar, Matthias Blume, and Jacob Donham. A consistent semantics of self-adjusting computation. *Journal of Functional Programming*, 23(3):249–292, 2013. doi: 10.1017/S0956796813000099.

[7] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte: Theory and practice. *Journal of Functional Programming*, 30:e11, 2020. doi: 10.1017/S0956796820000088.

[8] World Wide Web Consortium. PROV-DM: The PROV data model. W3c recommendation, World Wide Web Consortium, 2013. URL https://www.w3.org/TR/2013/REC-prov-dm-20130430/. Edited by Luc Moreau and Paolo Missier; 30 April 2013.

[9] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259. URL https://doi.org/10.1145/363235.363259.

[10] LangChain. LangGraph documentation: Persistence. https://docs.langchain.com/oss/python/langgraph/persistence, 2026. Accessed 19 October 2026. Experiment pins LangGraph 1.2.16.

[11] LangChain. LangGraph documentation: Use time-travel. https://docs.langchain.com/oss/python/langgraph/use-time-travel, 2026. Accessed 19 October 2026.

[12] Razeen A Rasheed, Somnath Banerjee, Animesh Mukherjee, and Rima Hazra. From fluent to verifiable: Claim-level auditability for deep research agents. arXiv preprint arXiv:2602.13855, 2026. URL https://arxiv.org/abs/2602.13855.

[13] Rui Meng, Bhavana Dalvi Mishra, Jiefeng Chen, Chun-Liang Li, Palash Goyal, Mihir Parmar, Yiwen Song, Yale Song, Rajarishi Sinha, Parthasarathy Ranganathan, Burak Gokturk, Jinsung Yoon, and Tomas Pfister. ScientistOne: Towards human-level autonomous research via chain-of-evidence. arXiv preprint arXiv:2605.26340, 2026. URL https://arxiv.org/abs/2605.26340.

[14] Hafumi Nishi. ClaimGarden: Update-aware claim-state control for AI scientist workflows. ICML 2026 AI for Science Workshop, 2026. URL https://openreview.net/forum?id=Y8INhipPQe. AI Scientist Proposal Competition track.

[15] Daniel Golovin, Andreas Krause, and Debajyoti Ray. Near-optimal bayesian active learning with noisy observations. In J. Lafferty, C. Williams, J. Shawe-Taylor, R. Zemel, and A. Culotta, editors, *Advances in Neural Information Processing Systems*, volume 23, pages 766–774. Curran Associates, Inc., 2010. URL https://arxiv.org/abs/1010.3091.

[16] David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550, Online, 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.609. URL https://aclanthology.org/2020.emnlp-main.609/.

[17] Pepa Atanasova, Jakob Grue Simonsen, Christina Lioma, and Isabelle Augenstein. Fact checking with insufficient evidence. *Transactions of the Association for Computational Linguistics*, 10:746–763, 2022. doi: 10.1162/tacl_a_00486. URL https://aclanthology.org/2022.tacl-1.43/.

[18] Qisheng Hu, Quanyu Long, and Wenya Wang. Decomposition dilemmas: Does claim decomposition boost or burden fact-checking performance? In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6313–6336, Albuquerque, New Mexico, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.320. URL https://aclanthology.org/2025.naacl-long.320/.

[19] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. Distribution-free, risk-controlling prediction sets. *Journal of the ACM*, 68(6):1–34, 2021. doi: 10.1145/3478535. URL https://doi.org/10.1145/3478535.

[20] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 16(4):494–591, 2023. doi: 10.1561/2200000101. URL https://doi.org/10.1561/2200000101.

[21] statsmodels developers. The Datasets Package. https://www.statsmodels.org/stable/datasets/index.html, n.d. Dataset exports and provenance archived from version 0.15.0. Accessed 19 October 2026.